CS535: Algorithm Design and Analysis

AVL Trees

Prof. Rohit Pal Singh

Department of Computer Science

University of Wisconsin, Milwaukee

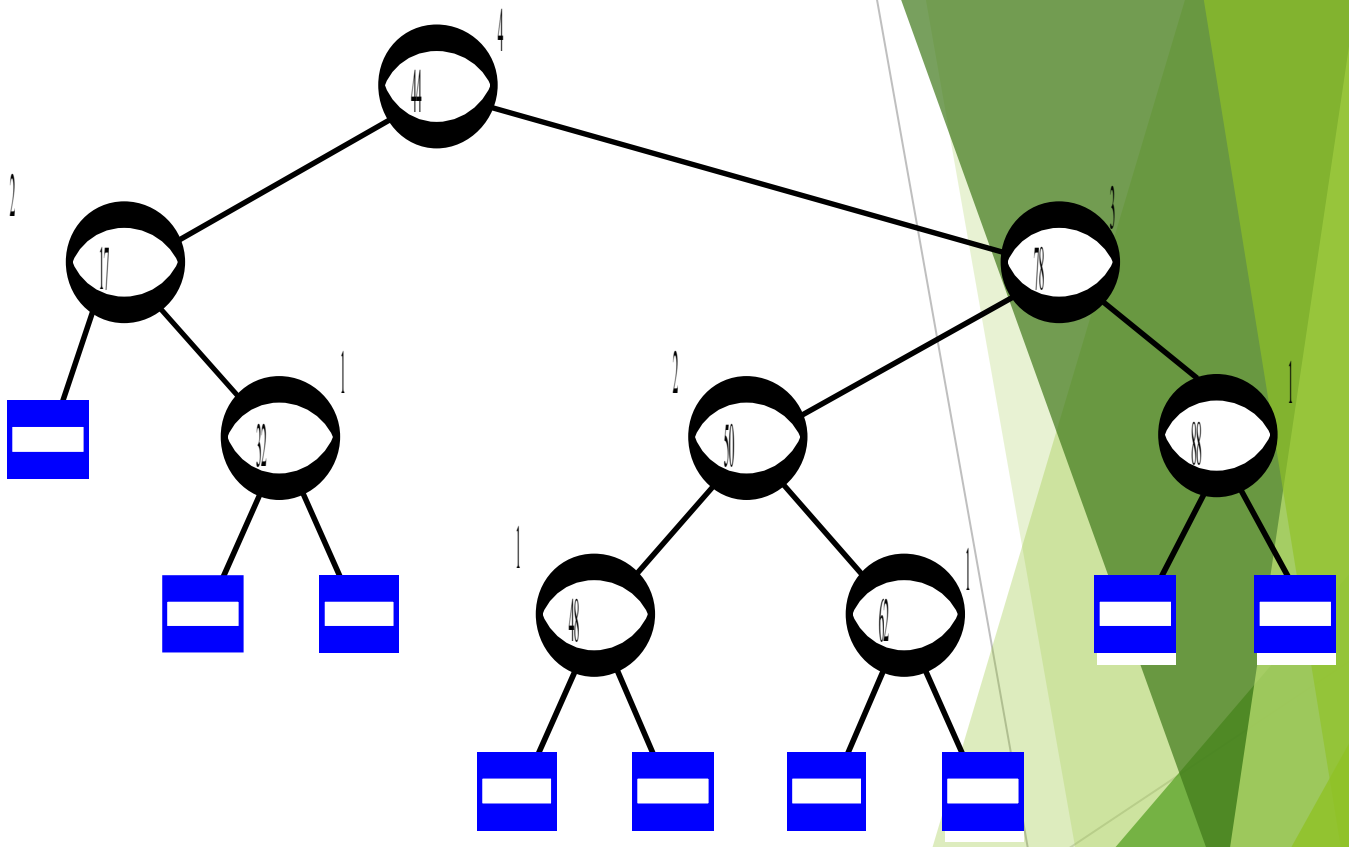
October 2, 2024

Readings: Chapter 3.2

AVL Tree Definition

- AVL trees are balanced.
- An AVL Tree is a ***binary search tree*** such that:

  for every internal node v of T, the *heights of the children of v can differ by at most 1*.



An example of an AVL tree where the heights are shown next to the nodes:

Height of an AVL Tree

- **Fact**: The ***height*** of an AVL tree storing n keys is O(log n).

- **Proof**: Let us bound **n(h):** the minimum number of internal nodes of an AVL tree of height h.

  We easily see that n(1) = 1 and n(2) = 2

  For h > 2, an AVL tree of height h contains the root node, one AVL subtree of height h-1 and another of height h-2.

  That is,

  n(h) = 1 + n(h-1) + n(h-2)

  Knowing n(h-1) > n(h-2), we get n(h) > 2*n(h-2). So

  n(h) > 2*n(h-2), n(h) > 4*n(h-4), n(h) > 8*n(n-6), … (by induction),

  n(h) > 2^{i}*n(h-2i)

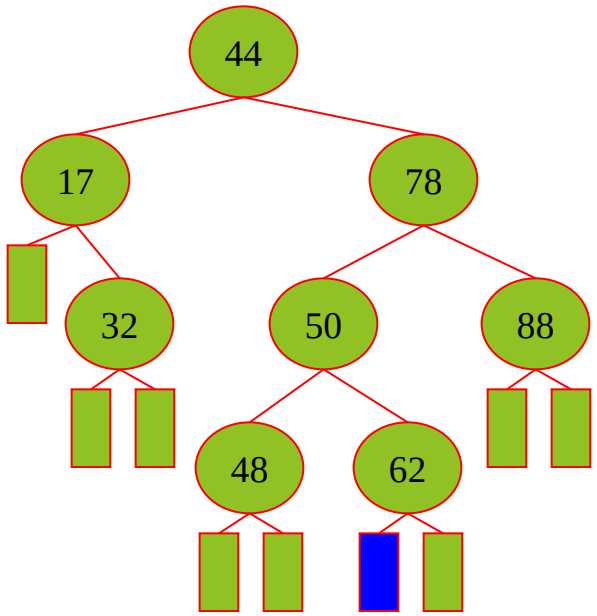
  Solving the base case we get: n(h) > $2^{h/2-1}$

  Taking logarithms: h < 2log n(h) +2

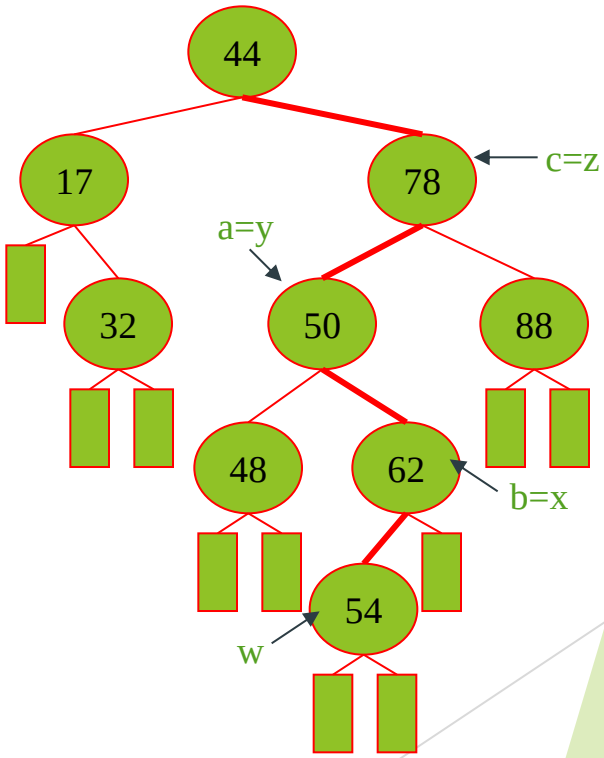
  Thus the height of an AVL tree is O(log n)

Insertion in an AVL Tree

- Insertion is as in a binary search tree
- Always done by expanding an external node.
- Example: Insert(54)



before insertion



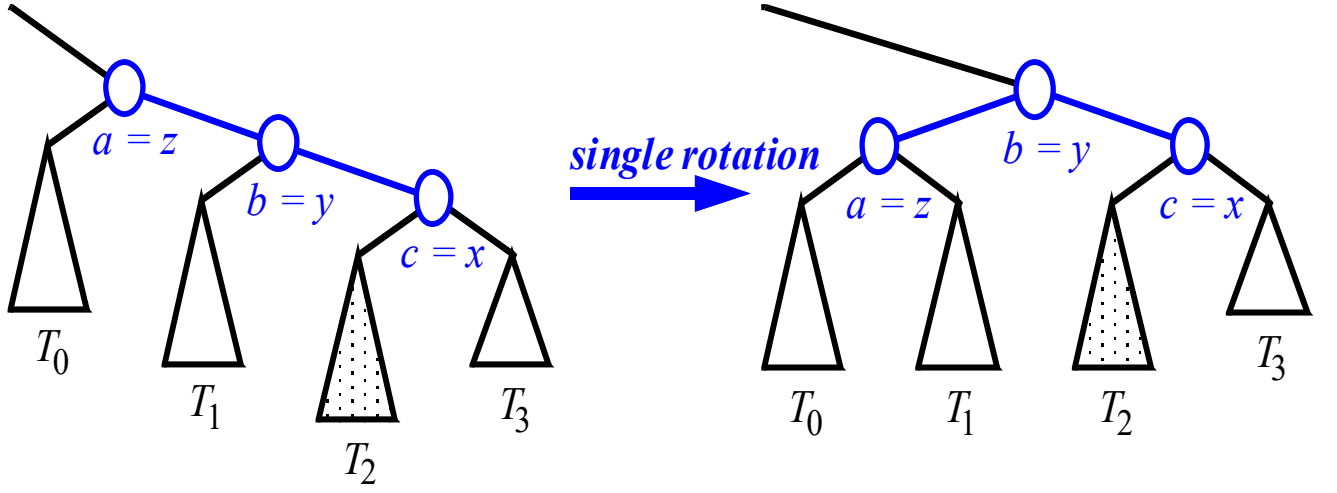
after insertion

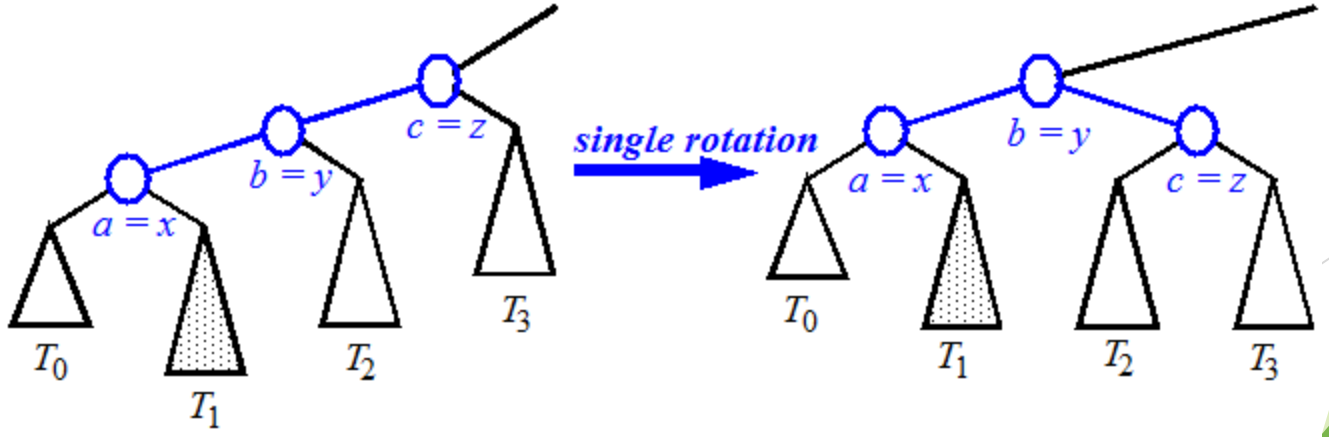
Trinode Restructuring

- Must keep two properties:
  - Binary search tree (in-order traversal)
  - Height-balance property
- The height changes only on path from the new node **w** to the root
- Start from **w** along this path, find the first un-balanced node **z**, then one of its children, denoted by **y**, with higher height (**y** must be the ancestor of **w**), finally one of **y**'s children, denoted by **x**, with higher height
- let (a, b, c) be an in-order listing of x, y, z, so that $a \leq b \leq c$
- Four cases to arrange a, b, c

Restructuring (as Single Rotations)

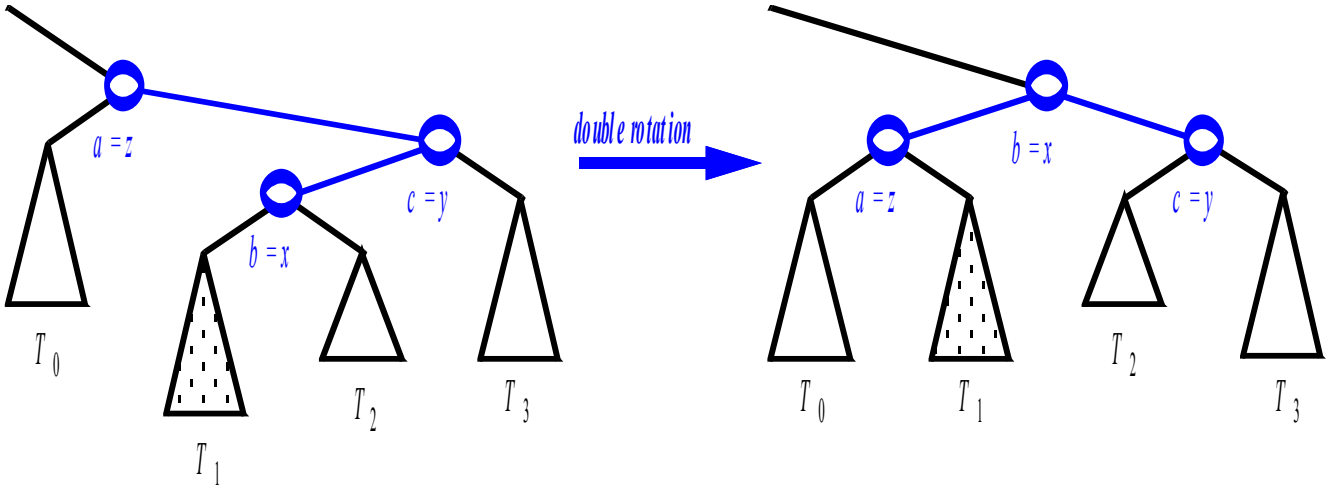
- Single Rotations:

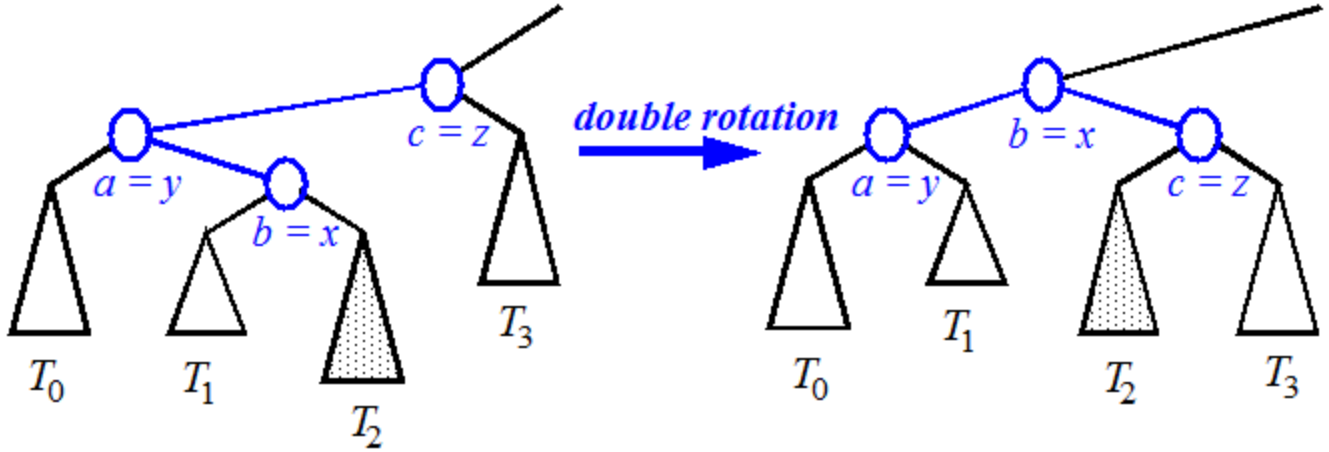




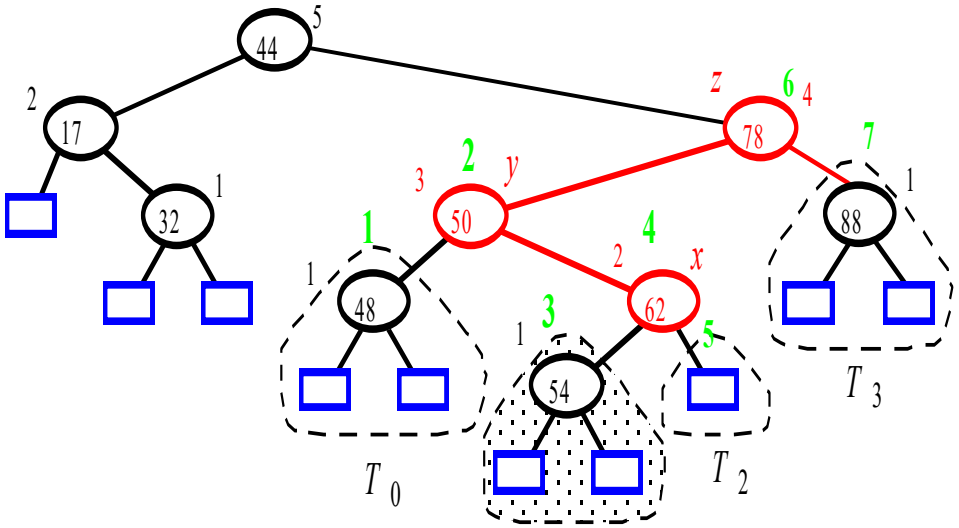
Restructuring (as Double Rotations)

- double rotations:

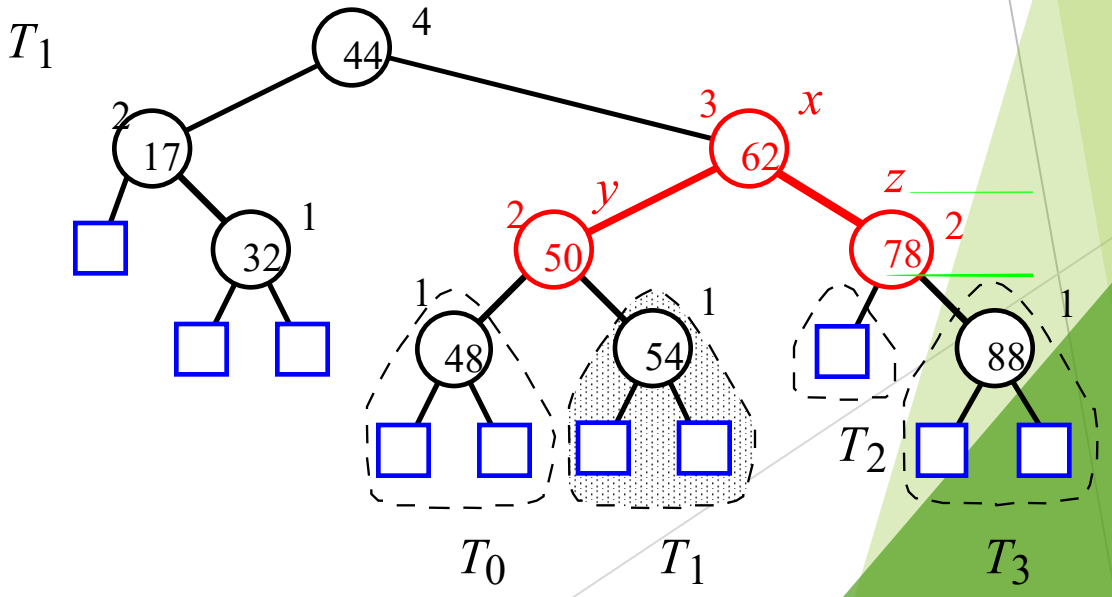




Insertion Example



unbalanced

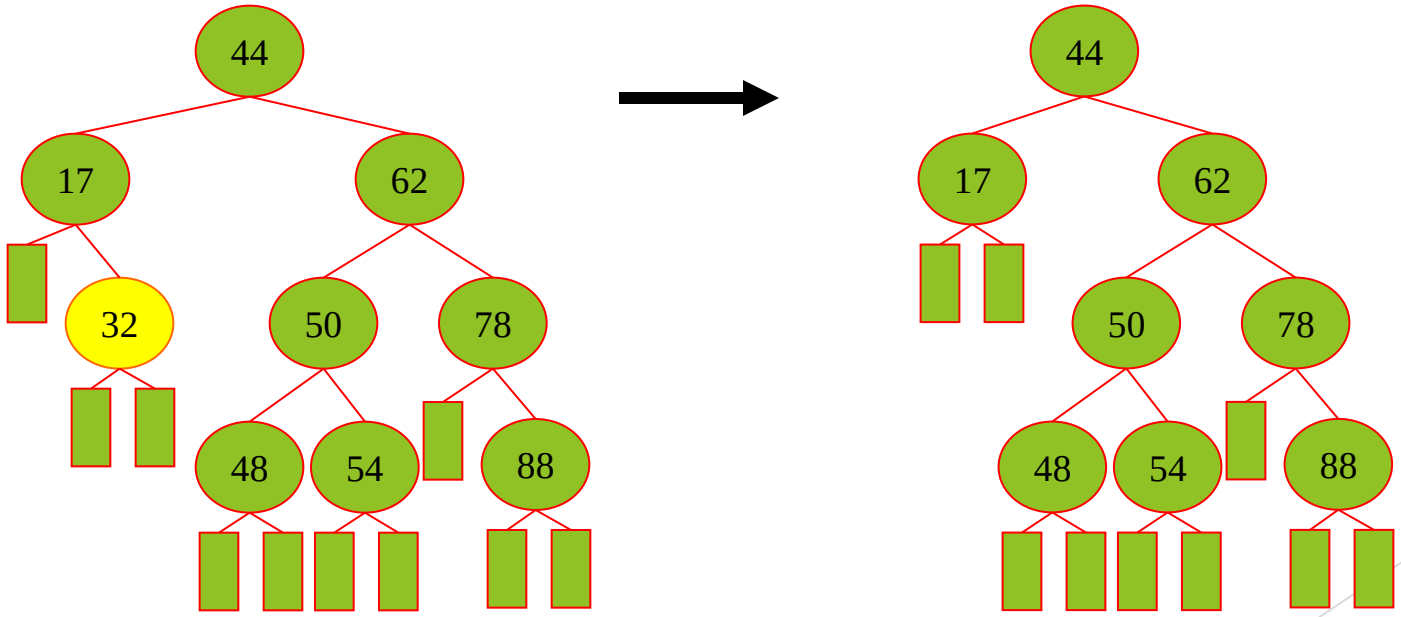


balanced

Removal in an AVL Tree

- Removal begins as in a binary search tree, which means the node removed will become an empty external node. Its parent, w, may cause an imbalance (the path from w to the root). However, at most one node on this path is unbalanced.

- Example:

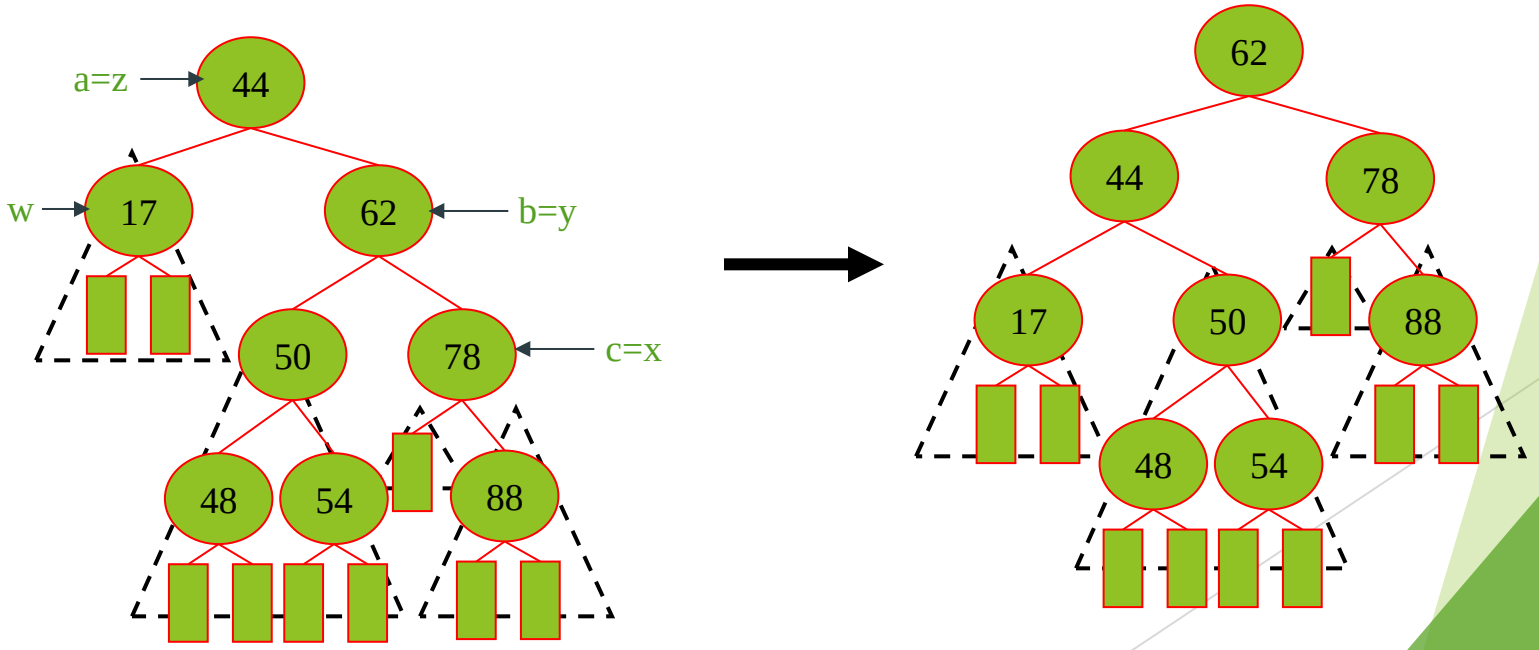


before deletion of 32

after deletion

Rebalancing after a Removal

- Let z be the unbalanced node encountered while travelling up the tree from w. Also, let y be the child of z with the larger height (y must not be an ancestor of w), and let x be the child of y with the larger height (otherwise, choose x to be the same side of y).
- We perform the restructuring procedure to restore balance at z.
- As this restructuring may upset the balance of another node higher in the tree, we must continue checking for balance until the root of T is reached (consider, for example, 48 and 54 are both external nodes)



Running Times for AVL Trees

- a single restructuring operation is O(1)
  - using a linked-structure binary tree
- **find** is O(log n)
  - height of tree is O(log n), no restructures needed
- **insert** is O(log n)
  - initial find is O(log n)
  - Restructuring up the tree, maintaining heights is O(log n)
- **remove** is O(log n)
  - initial find is O(log n)
  - Restructuring up the tree, maintaining heights is O(log n)